

grow_symmetric2.c

Growing 3-Fold Symmetric Hat Clusters from the $v[12]$ Pinwheel

MyHatTileStuff/grow_symmetric2.c — Technical Reference

June 15, 2026

Purpose

`grow_symmetric2` enumerates hat-tile clusters that grow ring-by-ring around a fixed *3-hat pinwheel* seed: three hat tiles sharing the single vertex $v[12]$ of the base hat cycle, placed at relative orientations 0, 120, and 240. After growing N concentric rings the program keeps only those completions whose outer boundary is invariant under a 120 rotation around the shared vertex, outputting them as a `.dat` record file and (optionally) an SVG grid.

The program answers the question: *can a 3-fold rotationally symmetric hat cluster centred at vertex $v[12]$ be extended indefinitely, or does the RL0 vertex-arc rulebook force a dead end?*

Background

The Tetrille Lattice and Hat Tile

The **tetrille** lattice tiles the plane with equilateral triangles and squares, producing vertices of three valences: $v = 6$ (hexagonal, shared by two squares and two triangles), $v = 4$ (square, shared by one square and two triangles), and $v = 3$ (triangular, shared by one square and one triangle). A lattice point is an integer triple (v, x, y) where v carries the valence tag; arithmetic across valence tags requires an explicit lifting formula.

The hat tile is a 14-vertex polygon whose boundary cycle in `hat.tile` reads:

$v[0] = (6, 0, 0)$	$v[1] = (4, -1, 0)$	$v[2] = (3, -1, 0)$
$v[3] = (4, -1, -1)$	$v[4] = (6, 0, -1)$	$v[5] = (4, 0, -3)$
$v[6] = (3, -1, -2)$	$v[7] = (4, 1, -3)$	$v[8] = (3, 0, -2)$
$v[9] = (4, 1, -2)$	$v[10] = (6, 1, -1)$	$v[11] = (4, 2, -1)$
$v[12] = (3, 1, -1)$	$v[13] = (4, 1, -1)$	

Choosing the Pinwheel Centre

A 3-hat pinwheel requires a vertex p with interior angle 120 inside the hat. Three of the hat's 14 vertices satisfy this: $v[0]$, $v[6]$, and $v[12]$. They differ in two important ways.

Vertex	Coord	Valence	RL0 status	Notes
$v[0]$	$(6, 0, 0)$	hexagonal	valid	Tetrille coordinate origin; fixed point of all lattice rotations; no translation needed for the symmetry check. Dead end at level 3 (51-tile cluster, no further symmetric extension).
$v[6]$	$(3, -1, -2)$	triangular	forbidden	Geometrically valid 120 angle, but the RL0 vertex-arc dictionary prohibits three hats meeting there in a pinwheel arrangement. Returns 0 completions even at level 1.
$v[12]$	$(3, 1, -1)$	triangular	valid	The centre used by this program. Not the origin, so a translation correction is required after each 60-lattice rotation. Gives an open cluster: 2, 4, 10 symmetric completions at levels 1, 2, 3.

Rotation Around a Non-Origin Centre

The Lattice Rotation Primitive

`cycle_transform_lattice(src, dst, lattice, t)` applies the *origin-centred* 60°-step lattice rotation

$$R_t : (x, y) \mapsto (-y, x + y)^t$$

to every vertex of `src`. The transform is exact on all three valence systems. A single application with $t = 1$ rotates by 60; $t = 2$ rotates by 120; $t = 4$ rotates by 240. The rotation centre is always *the lattice origin* $(v, 0, 0)$.

Rotating Around an Arbitrary Centre

To rotate a cycle by 120 around a point C that is *not* the origin, the standard conjugation trick is used:

$$\text{Rot}_{C,120}(\mathbf{p}) = C + R_2(\mathbf{p} - C) = R_2(\mathbf{p}) + \underbrace{(C - R_2(C))}_{\Delta}.$$

The displacement $\Delta = C - R_2(C)$ is a fixed integer vector that depends only on C . Because the tetrille $v = 3$ system is closed under the `tetrille_translate_cycle` operation (which works in the coarser $v = 6$ units), Δ must be representable in $v = 6$ units.

Computing the Translation Constants

Translation computation in build_symmetric_seed

The program computes Δ once at startup by applying the $t = 2$ rotation to a one-vertex cycle containing just $C = v[12]$, then computing the displacement back to C in the $v = 6$ coordinate system.

`tetrille_delta_to_6(valence, dx, dy, &m, &n)` converts an integer displacement (dx, dy) (in the coordinate system of a vertex with the given valence) into $v = 6$ lattice units (m, n) . For a $v = 3$ vertex: $a = dx - dy$, $b = dx + 2dy$; then $m = a/3$, $n = b/3$. The function returns 0 if dx, dy are not exactly representable, which would indicate an invalid centre.

For $v[12] = (3, 1, -1)$: after $t = 2$ the point maps to $(3, 0, 1)$, giving $dx = 1 - 0 = 1$, $dy = -1 - 1 = -2$. The formula yields $m_6 = +1$, $n_6 = -1$.

```
Coord center = tile->base.v[CENTER_VERTEX];
/* 12 = CENTER_VERTEX */

/* 1-vertex cycle to find where C maps */
Cycle tmp, rtmp;
memset(&tmp, 0, sizeof(tmp));
tmp.n = 1; tmp.v[0] = center;
cycle_transform_lattice(&tmp, &rtmp,
                      tile->lattice, 2);

int dx = center.x - rtmp.v[0].x; /* +1 */
int dy = center.y - rtmp.v[0].y; /* -2 */

tetrille_delta_to_6(center.v, dx, dy,
                  &g_rot_m6, &g_rot_n6);
/* g_rot_m6 = +1, g_rot_n6 = -1 */
```

The global pair `(g_rot_m6, g_rot_n6)` is used in every subsequent call to `has_120_symmetry`.

Seed Construction

The Three-Hat Pinwheel

build_symmetric_seed — overview

Three hat cycles are built at angles 0, 120, and 240:

- `rot[0] = tile->base` (orientation $t = 0$)
- `rot[1]`: obtained by `cycle_transform_lattice(base, t=2)`
- `rot[2]`: obtained by `cycle_transform_lattice(base, t=4)`

After applying the lattice rotation, $v[12]$ moves to a different position. The seed assembly step (below) searches over all (be, te) pairs to find the unique attachment of `rot[k]` that places its $v[12]$ back on the shared centre C .

```
Cycle rot[SEED_HATS];
rot[0] = tile->base;
cycle_transform_lattice(&tile->base,
                      &rot[1], tile->lattice, 2); /* 120° */
cycle_transform_lattice(&tile->base,
                      &rot[2], tile->lattice, 4); /* 240° */

/* poly[0] = first hat alone */
poly[0].cycle_count = 1;
poly[0].cycles[0] = rot[0];
hats[0] = rot[0];
```

Attaching Hat 2 and Hat 3

Attachment loop for $k = 1, 2$

For each subsequent hat ($k=1$ and $k=2$) the program iterates over every boundary edge be of the growing polygon and every edge index te of $rot[k]$. Three conditions must all hold for the placement to be accepted:

1. `try_attach_tile_poly_ex` succeeds (the tile can be placed without overlap).
2. The resulting polygon has no holes (`cycle_count == 1`).
3. The placed tile's vertex 12 coincides with the shared centre: `aligned.v[CENTER_VERTEX] == center`.

The third condition is what enforces the “three hats all touching at $v[12]$ ” constraint. Without it, the loop would accept any non-overlapping attachment of the rotated tile.

```
Edge edges[512];
for (int k = 1; k < SEED_HATS; k++) {
    int ok = 0;
    int ec = build_boundary_edges(
        &poly[k-1], edges);
    for (int be = 0; be < ec && !ok; be++)
        for (int te = 0;
            te < rot[k].n && !ok; te++) {
            Cycle aligned;
            if (!try_attach_tile_poly_ex(
                &poly[k-1], &rot[k],
                tile->lattice, be, te,
                &poly[k], &aligned)) continue;
            if (poly[k].cycle_count > 1) continue;
            /* KEY: v[12] must stay at centre */
            if (!coord_eq(
                aligned.v[CENTER_VERTEX],
                center)) continue;
            hats[k] = aligned;
            ok = 1;
        }
}
```

For the canonical hat tile the search succeeds at ($be = 10, te = 13$) for hat 2 and ($be = 17, te = 13$) for hat 3, producing a 30-vertex outer boundary.

BComp1 State Initialisation

Once the three tile cycles are assembled into `poly[2]` (a hole-free polygon) and `hats[0..2]`, a `BComp1Record` is filled in and passed to `bcomp1_state_from_record` to create the initial growth state. Three fields are mandatory for this call to succeed:

```
BComp1Record rec;
memset(&rec, 0, sizeof(rec));
rec.have_boundary = 1; /* outer polygon */
rec.have_tiles    = 1; /* tile list */
rec.have_center   = 1; /* seed boundary */
rec.center        = seed_center; /* = poly[2].cycles[0] */
rec.boundary      = poly[SEED_HATS - 1];
rec.tiles_count   = SEED_HATS;
for (int k = 0; k < SEED_HATS; k++) rec.tiles[k] = hats[k];
return bcomp1_state_from_record(&rec, seed);
```

Note: `have_center` must be set or `bcomp1_state_from_record` returns 0 silently. The `center` field stores the outer boundary of the 3-hat seed; it is not the abstract rotation centre C but the *cycle* used by the RL1 engine to track the boundary of the innermost “supertile”.

The RL0 / RL1 Growth Engine

BComp1Context

BComp1Context — what is loaded

`bcomp1_context_init(&ctx, tile_path, remembrance_path, deletions_path)` loads three data sources into a single context object:

tile_path
The hat tile definition file (`preferences/focus.tile`), parsed into `ctx.tile` including all variant cycles.

remembrance_path
The RL0 *remembrance* table (`data/rl0/remembrance.dat`): the dictionary of valid vertex-arc completions at every lattice valence. Used by `attach0_force_live_closure` and the BComp1 growth engine to determine which tile placements are globally consistent.

deletions_path
The RL0 *deletions* table (`data/rl0/deletions.dat`): vertex-arc completions that have been proved inconsistent and must be excluded.

Together the remembrance and deletions tables constitute the **RL0 rulebook** — the first-order local constraint set for valid hat tilings.

```
BComp1Context ctx;
bcomp1_context_init(&ctx,
    "preferences/focus.tile",
    "data/rl0/remembrance.dat",
    "data/rl0/deletions.dat");

/* ctx.tile: loaded Tile struct */
/* ctx.map: RL0 forget/arc map */
```

Forced Deterministic Closure

attach0_force_live_closure

After the 3-hat pinwheel seed is built, the program attempts a *forced ring*: it repeatedly scans every boundary vertex of the current polygon and, wherever the RL0 rulebook leaves only one possible tile placement, makes that placement deterministically. This continues until no more forced moves remain.

The function returns 1 (ok) in two situations:

- No forced moves exist from the start (0 steps), or
- All forced moves were made and no contradiction was reached.

It returns 0 only when the RL0 rules produce a contradiction — a vertex with no valid completion at all.

For the $v[12]$ pinwheel, the forced closure completes with `closure_steps=0` and `unresolved=30`: the 3-hat seed has no deterministic forced moves. Every boundary vertex still has multiple valid completions. This is fundamentally different from the $v[0]$ pinwheel (where forced closure adds 9 tiles in 6 steps and resolves all boundary vertices), and explains why the $v[12]$ seed yields multiple symmetric completions at every ring level.

```
Attach0Stats astats;
Attach0ClosureStats cstats;
attach0_stats_init(&astats);
attach0_closure_stats_init(&cstats);

int ok = attach0_force_live_closure(
    &seed.poly,
    &ctx.tile,
    seed.tiles,
    &seed.tile_count,
    &ctx.map,
    FORCE_STEPS, /* = 1024 */
    &astats,
    &cstats);

/* v[12] result:
   ok=1, closure_steps=0, unresolved=30
   => no forced moves, not a dead end */
```

Ring-by-Ring Growth with BComp1

bcomp1_complete_state — growth options

`bcomp1_complete_state(&ctx, &seed, ¢er, &opts, &result)` grows N concentric rings from the seed state. A “ring” is one additional layer of RL0-consistent tile placements. The function exhaustively enumerates all completions up to the requested depth and collects them into `result.records`.

The three key options are:

`depth = N`

Number of rings to add.

`live_only = 1`

Discard any completion that the RL0 rulebook marks as a dead end (a state with no valid infinite extension).

`collect_records = 1`

Store every surviving completion as a `BComp1Record` in `result.records`.

Without `live_only=1` the count at level 3 would be much larger; the flag prunes early those clusters that cannot be part of a valid hat tiling.

```
BComp1Options opts;
bcomp1_options_default(&opts);
opts.depth          = level; /* --level N */
opts.collect_records = 1;
opts.live_only       = 1;

BComp1Result result;
bcomp1_complete_state(
    &ctx, &seed, &center,
    &opts, &result);

/* result.records.count = total live
   completions at requested depth */
```

Symmetry Filtering

The 120° Invariance Test

has_120_symmetry

A completion’s outer boundary is **120°-symmetric** if rotating it by 120 around $C = v[12]$ produces the same cycle (up to a cyclic shift of the start vertex). This is the defining property of the pinwheel symmetry group $\mathbb{Z}/3\mathbb{Z}$.

The test has two steps:

1. Apply the origin-centred $t = 2$ lattice rotation to the boundary cycle: `cycle_transform_lattice(c, &rotated, lattice, 2)`.
2. Translate the result by `(g_rot_m6, g_rot_n6)` to undo the displacement of C under the rotation: `tetrille_translate_cycle(&rotated, g_rot_m6, g_rot_n6)`.

The two steps together implement $\text{Rot}_{C,120}$. The resulting cycle `rotated` is compared to the original `c` by `cycles_match_cyclic`, which allows any cyclic shift.

```
static int has_120_symmetry(
    const Poly *boundary, int lattice)
{
    if (boundary->cycle_count != 1)
        return 0;
    const Cycle *c = &boundary->cycles[0];
    Cycle rotated;
    /* step 1: rotate around origin */
    cycle_transform_lattice(
        c, &rotated, lattice, 2);
    /* step 2: translate so C is fixed */
    tetrille_translate_cycle(
        &rotated, g_rot_m6, g_rot_n6);
    /* compare allowing cyclic shift */
    return cycles_match_cyclic(c, &rotated);
}
```

Cyclic Matching

`cycles_match_cyclic`

Two cycles represent the same geometric polygon if they have the same vertices in the same order but possibly starting at a different point. The function searches all cyclic shifts s of b and checks whether any shifted b agrees vertex-for-vertex with a . Both vertex count and all coordinate triples (v, x, y) must match exactly.

```
static int cycles_match_cyclic(
    const Cycle *a, const Cycle *b)
{
    if (a->n != b->n) return 0;
    for (int s = 0; s < b->n; s++) {
        if (!coord_eq(b->v[s], a->v[0]))
            continue; /* wrong start */
        int ok = 1;
        for (int i = 1; i < a->n && ok; i++)
            if (!coord_eq(a->v[i],
                          b->v[(s+i) % b->n]))
                ok = 0;
        if (ok) return 1;
    }
    return 0;
}
```

In-Place Record Filtering

After `bcomp1_complete_state` fills `result.records`, the program compacts the array in-place, overwriting non-symmetric entries:

```
size_t total = result.records.count;
size_t sym_count = 0;
for (size_t i = 0; i < total; i++) {
    if (has_120_symmetry(&result.records.items[i].boundary, ctx.tile.lattice))
        result.records.items[sym_count++] = result.records.items[i];
}
result.records.count = sym_count;
```

This avoids any dependency on a non-public `push/init/clear` API for `BComp1RecordVec`: the filter works entirely through the public `items[]` and `count` fields.

Helper Functions

Collecting Interior (Hidden) Vertices

rebuild_hidden and collect_verts

A BComp1Record carries a list of *hidden vertices*: lattice points that are interior to the cluster (touched by at least one tile but not lying on the outer boundary). The count of hidden vertices serves as the RL1 *level* field, a measure of cluster depth.

collect_verts builds the union of all vertex sets from all tile cycles (deduplicating by linear scan). rebuild_hidden then subtracts the boundary vertices (from build_boundary_vertices) to obtain only the interior points. Results are sorted by the lexicographic order (v, x, y) for reproducibility.

```
/* Collect all tile vertices */
static int collect_verts(
    const Cycle *tiles, int n, Coord *out)
{
    int c = 0;
    for (int t = 0; t < n; t++)
        for (int i = 0; i < tiles[t].n; i++)
            if (!coord_in(out, c, tiles[t].v[i]))
                out[c++] = tiles[t].v[i];
    return c;
}

/* Hidden = all_verts \ boundary_verts */
static int rebuild_hidden(
    const Poly *p, const Cycle *tiles,
    int n, Coord *hidden)
{
    Coord all[BCOMP1_MAX_COORDS];
    Coord bnd[BCOMP1_MAX_COORDS];
    int ac = collect_verts(tiles, n, all);
    int bc = build_boundary_vertices(p, bnd);
    int hc = 0;
    for (int i = 0; i < ac; i++)
        if (!coord_in(bnd, bc, all[i]))
            hidden[hc++] = all[i];
    qsort(hidden, hc, sizeof(Coord),
        coord_cmp);
    return hc;
}
```

Assembling a BComp1Record from a State

make_record(&state, ¢er, &rec) constructs the full BComp1Record needed to write a structured output file. It allocates a temporary buffer, calls rebuild_hidden to count and list interior vertices, then populates all mandatory fields:

```
r->have_center = r->have_boundary = r->have_hidden = r->have_tiles = 1;
r->level       = hc;                      /* hidden vertex count = depth metric */
r->center      = *center;
r->boundary    = s->poly;
r->hidden_count = hc;
r->tiles_count  = r->tile_count = s->tile_count;
```

SVG Rendering

Coordinate Conversion

Tetrille coordinates (v, x, y) are mapped to Euclidean screen space using the basis vectors that the `hat.tile` lattice prescribes ($\sqrt{3}$ pre-computed as `g_r3`):

$$(x_s, y_s) = \begin{cases} \left(\frac{\sqrt{3}}{2}(x+y), \frac{1}{2}(y-x) \right) & v = 6 \\ \left(\frac{\sqrt{3}}{4}(x+y), \frac{1}{4}(y-x) \right) & v = 4 \\ \left(\frac{x+\frac{1}{2}y}{\sqrt{3}}, \frac{1}{2}y \right) & v = 3 \end{cases}$$

SVG y increases downward, so the draw call negates y_s : a tile vertex at (x_s, y_s) appears at SVG position $(o_x + \text{SC} \cdot x_s, o_y - \text{SC} \cdot y_s)$ where (o_x, o_y) is the per-cell offset and `SC = 24` is the pixel scale factor.

Grid Layout

A global bounding box is computed over all tile vertices in all output records, so every grid cell uses the same scale. The grid dimensions (rows $\times$ columns) are chosen to minimise the deviation from a 16 : 9 aspect ratio:

```
for (int r = 1; r <= n; r++) {
    int c2 = (n + r - 1) / r;
    double sc2 = fabs((double)c2 * cw / ((double)r * ch) - 16.0 / 9.0);
    if (sc2 < best) { best = sc2; rows = r; cols = c2; }
}
```

Colour Coding

Tiles are drawn in a fixed layer order (grown tiles first, seed tiles on top, centre outline last) with the following colour scheme:

Role	Colour	Hex
Seed hat 0 (base orientation)	 Amber	#f39c12
Seed hat 1 (120° rotation)	 Red	#e74c3c
Seed hat 2 (240° rotation)	 Purple	#8e44ad
Grown tiles (ring 1, 2, ...)	 Light blue	#a8d8ea
Seed boundary outline	 Dark red	#c0392b

The seed boundary outline (the outer boundary of the 3-hat seed itself) is drawn with no fill and a thick stroke, giving a visual landmark for the pinwheel centre across all completions.

Main Pipeline

The main function executes the following steps in order:

1. **Argument parsing.** Accepts `-level N`, `-tile`, `-remembrance`, `-deletions`, `-out-dir`, and `-svg` flags.
2. **Context initialisation.** `bcomp1_context_init` loads the tile and RL0 rule tables.
3. **Seed construction.** `build_symmetric_seed`: computes rotation constants, builds the three rotated hat cycles, searches for the shared- $v[12]$ attachment, assembles the `BComp1State`.
4. **Forced closure.** `attach0_force_live_closure` applies all deterministic RL0 moves (0 for the $v[12]$ seed).
5. **Supertile record.** `make_record` + `bcomp1_print_record` writes the seed cluster to `data/sym12/supertile.dat`.
6. **Level- N growth.** `bcomp1_complete_state` with `depth=N`, `live_only=1`, `collect_records=1`.
7. **Symmetry filter.** In-place compaction retaining only records whose boundary satisfies `has_120_symmetry`.
8. **Output.** Sorted records written to `data/sym12/sym_completions_levelN.dat`; SVG rendered if `-svg` was given.

Results

Running `./bin/grow_symmetric2` with the default canonical hat tile and RL0 rule tables produces:

Level	Total live	Symmetric	Remarks
1	4	2	Two distinct ways to add a first ring with 120° symmetry.
2	24	4	Four symmetric clusters after two rings.
3	700	10	Ten symmetric clusters; branching is widening.

Diagnostic output to `stderr`:

```
Symmetric 3-hat seed (center=v[12]=3(1,-1), rot_m6=1 rot_n6=-1, tiles=3)
grow_symmetric2 center=v[12] level=3
forced tiles=3 closure_steps=0 unresolved=30
completions=700 symmetric=10 -> data/sym12/sym_completions_level3.dat
```

Contrast with the $v[0]$ pinwheel. `grow_symmetric` (the companion program centred at $v[0]$) gives 1 symmetric completion at levels 1 and 2, and 0 at level 3: the cluster is a genuine dead end. The $v[12]$ pinwheel has no such limitation — the growing count of completions at each level strongly suggests the configuration can be extended to a valid infinite hat tiling with 3-fold rotational symmetry.

Architecture Summary

```
main()
+-- bcomp1_context_init(tile, remembrance, deletions)
|   loads Tile + RLO rule tables into BComp1Context
|
+-- build_symmetric_seed(ctx.tile, &seed, &center)
|   +-- center = tile->base.v[12] = (3,1,-1)
|   +-- cycle_transform_lattice(tmp, t=2) -> compute g_rot_m6=+1, g_rot_n6=-1
|   +-- rot[0]=base, rot[1]=t=2(base), rot[2]=t=4(base)
|   +-- for k=1,2: search (be,te) with aligned.v[12]==center
|   +-- bcomp1_state_from_record(&rec, &seed) [need have_center=1]
|
+-- attach0_force_live_closure(&seed, ...)
|   v[12] seed: 0 forced moves, 30 unresolved vertices (open config)
|
+-- make_record(&seed, &center, &supertile)
|   rebuild_hidden: all_verts \ boundary_verts, sorted
|   -> write data/sym12/supertile.dat
|
+-- bcomp1_complete_state(&ctx, &seed, &center, depth=N, live_only=1)
|   exhaustive ring-by-ring growth guided by RLO rulebook
|   result.records.count = total live completions
|
+-- symmetry filter (in-place compaction)
|   for each record: has_120_symmetry(boundary)?
|       cycle_transform_lattice(t=2) + tetrille_translate_cycle(+1,-1)
|       cycles_match_cyclic (cyclic shift allowed)
|       result.records.count = symmetric count
|
+-- bcomp1_sort_records + bcomp1_print_record
|   -> write data/sym12/sym_completions_levelN.dat
|
+-- emit_svg (optional)
|   bbox over all records -> uniform scale SC=24
|   draw grown tiles (#a8d8ea), seed trio (orange/red/purple)
|   draw seed boundary outline (#c0392b, no fill)
|   grid layout: minimise deviation from 16:9
```

Usage

```
# Build
make grow_symmetric2

# Level 1 (default)
./bin/grow_symmetric2

# Level 3 with SVG output
./bin/grow_symmetric2 --level 3 --svg img/sym12_level3.svg

# Custom output directory and tile file
./bin/grow_symmetric2 --level 2 --out-dir data/my_run \
  --tile preferences/focus.tile \
  --remembrance data/rlo/remembrance.dat \
  --deletions data/rlo/deletions.dat
```

Output files written to out-dir (default data/sym12):

- supertile.dat — the 3-hat seed cluster as a BComp1Record.

- `sym_completions_levelN.dat` — all 120° -symmetric live completions at the requested ring depth, one record per line.
-

`MyHatTileStuff/grow_symmetric2.c` — part of PolyformTown. Compile this document with `pdflatex -shell-escape grow_symmetric2.tex` (requires `pygments` for minted).